

Reverse after swapping

To make it harder still for people to break your encryption, reverse the message after encrypting it with `swap_letters()`. To do this, you'll need to create two different functions – one to encrypt and one to decrypt.

Reverses the message once its letters have been swapped.

1 Encrypt function

The `encrypt()` function swaps the letters and then reverses the string. Type these lines under the `swap_letters()` function.

```
def encrypt(message):
    swapped_message = swap_letters(message)
    encrypted_message = ''.join(reversed(swapped_message))
    return encrypted_message
```

2 Decrypt function

Add this `decrypt()` function beneath the `encrypt()` function. It starts by reversing the encrypted message, and then uses `swap_letters()` to put the letters back in the right order.

```
def decrypt(message):
    unreversed_message = ''.join(reversed(message))
    decrypted_message = swap_letters(unreversed_message)
    return decrypted_message
```

Undo the reverse action of the encrypt function by reversing the message again.

This line puts the letters back in the right order.

3 Use the new functions

Now you need to update the infinite loop section of your program to use these functions instead of the `swap_letters()` function.



Don't forget to save your work.

```
while True:
    task = get_task()
    if task == 'encrypt':
        message = get_message()
        encrypted = encrypt(message)
        messagebox.showinfo('Ciphertext of the secret message is:', encrypted)
    elif task == 'decrypt':
        message = get_message()
        decrypted = decrypt(message)
        messagebox.showinfo('Plaintext of the secret message is:', decrypted)
    else:
        break
```

The new `encrypt()` function replaces `swap_letters()`.

The new `decrypt()` function replaces `swap_letters()`.